

A SOTA-inspired 3D Scene Occupancy Completion Pipeline on Real NYUv2 Depth Data

MSc Coursework Report

Langhui Huang
Author

Course: [Master Class 25/26](#)

Supervisor: Zhidong Xiao

Repository: github.com/NCCA/industry-project-HuangLanghui

Abstract

This report describes a pipeline for 3D scene occupancy completion built on real indoor depth data. The input is a single raw NYUv2 depth map. The system back-projects that depth into a 3D point cloud, voxelizes the cloud into a $64 \times 64 \times 64$ binary occupancy grid, and trains a small residual-attention 3D U-Net to predict a fuller occupancy volume. The training target is a proxy taken from NYUv2's in-painted depth rather than true 3D ground truth. Training was done from scratch on a single consumer GPU. On a held-out test set of 146 scenes the model reaches IoU 0.726, Precision 0.771, Recall 0.923, and F1 0.839. Because the input and target come from the same scene, a model could score well just by copying its input, so I also evaluate two non-learned baselines. The trained model improves IoU by 0.076 over a copy-input baseline and fills in occupancy that a morphological dilation cannot, which is the evidence that it is actually completing the scene. Six ablation studies cover the decode threshold, input missingness, point density, voxel resolution, noise filtering, and semantic priors, and show where the pipeline holds up and where it weakens. The work is a prototype on real RGB-D data rather than a benchmark method, and each result can be regenerated from the metrics JSON, figures, and PLY files in the repository.

Contents

1. Introduction	4
1.1 Problem and motivation	4
1.2 Aims and research questions	4
1.3 Contributions	4
2. Related Work	5
2.1 Semantic and geometric scene completion	5
2.2 NYUv2 and depth in-painting	5
2.3 Volumetric networks, residual/attention blocks, and overlap losses	5
2.4 Design alternatives considered	5
3. Problem Formulation	6
4. Method	6
4.1 Depth to point cloud	6
4.2 Voxelization	7
4.3 Input and proxy-target construction	7
4.4 Network architecture	7
4.5 Loss function	7
4.6 Metrics	8
5. Experimental Setup	8
6. Results	9
6.1 Main quantitative results	9
6.2 Completion versus copying (non-learned baselines)	10
6.3 Qualitative demonstration	10
6.4 What the completion mechanism does	11
7. Ablation Studies	12
7.1 Decode threshold	12
7.2 Robustness to input missingness	12
7.3 Partial-generation patterns	13
7.4 Point-cloud density	13
7.5 Voxel resolution	14
7.6 Noise filtering	15
7.7 Semantic-prior / label-assisted analysis	15
8. Discussion and Reflection	16
9. Limitations	16
10. Future Work	17
11. Conclusion	17
References	17

1. Introduction

1.1 Problem and motivation

A depth sensor sees only the visible surfaces of a room from one viewpoint, and even those surfaces come back with holes. The Kinect used to record NYUv2 drops large regions on dark, glossy, thin, or far-away objects. 3D scene completion takes this kind of partial, noisy observation and tries to infer a more complete occupancy of the scene. It matters for applications such as robotics, AR/VR, and indoor mapping, where a system needs some estimate of where solid matter is, even in places the sensor did not capture well.

1.2 Aims and research questions

The task in this coursework is this: given an incomplete occupancy volume built from a raw NYUv2 depth map, predict an occupancy volume that is closer to the fuller, in-painted-depth occupancy of the same scene. I set four questions to guide the work.

- **RQ1, feasibility.** Can a 3D network at single-GPU scale learn a useful mapping from raw to fuller occupancy on real NYUv2 depth, rather than on synthetic data?
- **RQ2, completion versus copying.** Since the input and target are the same scene, does the model really complete the volume or does it just repeat what it was given? This is the question that decides whether the task means anything.
- **RQ3, sensitivity.** How much does performance depend on the decode threshold, input degradation, point density, and voxel resolution?
- **RQ4, priors.** Do the NYUv2 2D labels help if I use them as a filtering prior?

1.3 Contributions

1. An end-to-end pipeline on real data (NYUv2 .mat $\rightarrow$ point cloud $\rightarrow$ voxel occupancy $\rightarrow$ 3D U-Net prediction $\rightarrow$ metrics, figures, and PLY export) that runs from a single config file. It trains and evaluates from scratch on one consumer GPU.
2. A residual-attention 3D U-Net of about 1.4M parameters, trained to IoU 0.726 and F1 0.839 on held-out NYUv2 scenes.
3. Two non-learned baselines, copy-input and dilation, that measure how much the trained model adds over trivial heuristics. These are the evidence for RQ2 (Section 6.2).
4. A single-command demo (demo.py) that shows, for each scene, the occupancy being filled in, together with TP/FP/FN error maps and colour-coded PLY clouds (Section 6.3).
5. Six ablation studies that show when and why the pipeline works, not just that it does (Section 7).

2. Related Work

2.1 Semantic and geometric scene completion

The main reference for this task is SSCNet (Song et al., 2017). It framed semantic scene completion from a single depth image and used a volumetric CNN over a flipped-TSDF encoding, trained on the synthetic SUNCG dataset and evaluated on NYU/NYUCAD. Later work, surveyed by Roldão, de Charette and Verroust-Blondet (2022), added things like two-

stream RGB and depth fusion, sparse convolutions, and coarse-to-fine decoders. All of these need dense volumetric 3D ground truth, which SUNCG can provide because it is synthetic. My setting is different on purpose. The NYUv2 labeled set has no per-voxel completed 3D ground truth, so I do not claim to complete truly occluded geometry. Instead I use a proxy target derived from depth (Section 3), and I state that limitation clearly throughout the report.

2.2 NYUv2 and depth in-painting

NYUv2 (Silberman et al., 2012) gives aligned RGB, raw Kinect depth (`rawDepths`, which has holes), a filled in-painted depth (`depths`), and dense 2D labels. The filling is the colorization-style method of Levin, Lischinski and Weiss (2004) that the official toolbox uses. I use this raw-to-filled relationship directly. The raw depth is the incomplete observation and the filled depth is a fuller proxy, and both come from the same real scene. Without synthetic 3D ground truth, this is the most honest version of “completion” I can set up on this dataset.

2.3 Volumetric networks, residual/attention blocks, and overlap losses

The network is built from standard, well-tested parts. The backbone is a U-Net encoder-decoder with skip connections (Ronneberger, Fischer and Brox, 2015) in its volumetric 3D form (Çiçek et al., 2016). It uses residual blocks for more stable training (He et al., 2016) and squeeze-and-excitation channel attention (Hu, Shen and Sun, 2018). The loss combines a soft Dice term (Milletari, Navab and Ahmadi, 2016) with class-weighted BCE to handle the heavy imbalance between empty and occupied voxels. When I call the model “SOTA-inspired” I mean this: known components put together at coursework scale, not a new benchmark method.

2.4 Design alternatives considered

Table 1 lists the main design choices, the options I considered instead, and the reason for each decision.

Decision	Chosen	Alternatives considered	Why the choice
Scene representation	Binary voxel occupancy	TSDF / point set / implicit field	Voxels are directly consumable by 3D convolutions and give an unambiguous IoU; TSDF needs signed-distance ground truth we do not have.
Target signal	In-painted-depth proxy	Synthetic SUNCG GT; multi-view fusion	NYUv2 labeled has no per-voxel 3D GT; SUNCG is synthetic (domain gap); no multi-view is available for the labeled subset.
Backbone	Residual-attention 3D U-Net	Plain 3D CNN; sparse conv; transformer	U-Net skips recover fine geometry; residual + SE add stability/selectivity cheaply; sparse/transformer 3D was out of scope for a single-GPU prototype.
Loss	Weighted BCE + soft Dice	BCE only; focal; Dice only	BCE alone collapses under class imbalance; Dice directly optimises the reported IoU/F1; the sum balances per-voxel and region objectives (validated in Section 7).

Table 1. Key design decisions and the alternatives that were considered and rejected.

3. Problem Formulation

Let $X \in \{0,1\}^{D \times H \times W}$ be the input occupancy grid built from the raw depth map, and $Y \in \{0,1\}^{D \times H \times W}$ the proxy target occupancy built from the in-painted depth of the same scene, with $D = H = W = 64$. The network f_θ predicts per-voxel logits $f_\theta(X)$; applying a sigmoid and a threshold τ yields the binary prediction

$$\hat{Y} = [\sigma(f_\theta(X)) \geq \tau].$$

Training minimises a class-weighted BCE + Dice loss (Section 4.5); evaluation reports IoU / Precision / Recall / F1 of $\hat{Y}$ against Y .

A note on the target. Y is a proxy, not real ground truth for hidden geometry. A single view only sees visible surfaces, so Y mainly adds back the holes in the visible surface that the raw depth lost, plus a thin dilation envelope. For this reason I do not claim to complete occluded regions. What I claim, and what I measure, is recovery of the fuller occupancy of the scene that was actually observed. Section 6.2 measures how much of $Y \setminus X$ the model recovers.

4. Method

4.1 Depth → point cloud

Every valid depth pixel (u, v) with depth Z is back-projected with the pinhole model using the NYUv2 aligned-RGB intrinsics ($f_x = 518.86$, $f_y = 519.47$, $c_x = 325.58$, $c_y = 253.74$):

$$x = (u - c_x) \cdot Z / f_x, \quad y = (v - c_y) \cdot Z / f_y, \quad z = Z$$

Depth values that are zero, NaN, or outside $[0.4 \text{ m}, 8.0 \text{ m}]$ are discarded; pixels are sub-sampled with `pixel_stride = 2` for speed (`src/nyuv2_scc/geometry.py`).

4.2 Voxelization

Points are clipped to fixed camera-space bounds $x \in [-3.2, 3.2]$, $y \in [-2.4, 2.4]$, $z \in [0.4, 8.0]$ metres and hashed into a $64 \times 64 \times 64$ grid stored in $[D, H, W] = [z, y, x]$ order. The resulting voxel size is anisotropic $\approx 0.075\text{--}0.12 \text{ m}$ (0.10 m in x , 0.075 m in y , 0.119 m in z). A single binary dilation (`dilate_iterations = 1`) closes 1-voxel gaps from sparse sampling so thin surfaces stay connected.

4.3 Input and proxy-target construction

Input occupancy comes from `rawDepths`; the proxy target from `depths`. During training only, the input is further degraded on the fly (`make_incomplete_input`): random voxel dropout (5%) plus one random cuboid cut-out, so the network has to fill in plausible missing geometry instead of memorising one fixed hole pattern. Validation and test inputs are never augmented, so the reported test numbers reflect the true raw-depth occupancy the sensor produced.

4.4 Network architecture

`ResidualAttentionUNet3D` (`src/nyuv2_scc/model.py`), $\sim 1.4\text{M}$ trainable parameters at `base_channels = 8`:

- **Encoder:** an input residual block at full resolution, then four down-stages ($\text{MaxPool3d} \times 2 \rightarrow \text{residual block}$), channel widths $8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$.
- **Residual block:** two $3 \times 3 \times 3$ convs + BatchNorm + ReLU with a skip connection, followed by an optional squeeze-and-excitation channel-attention gate (`use_attention`, toggled in the attention ablation).
- **Decoder:** four up-stages ($\text{ConvTranspose3d} \times 2 \rightarrow \text{concatenate the matching encoder skip} \rightarrow \text{residual block}$), mirroring the encoder widths back to 8.
- **Head:** a $1 \times 1 \times 1$ conv producing a single occupancy logit per voxel. There is no sigmoid inside the network; the sigmoid and threshold live in the loss and metric code.

4.5 Loss function

The training objective (`src/nyuv2_scc/losses.py`, `BCEDiceLoss`) is a weighted sum of two terms:

$$\text{loss} = \text{bce_weight} \cdot \text{BCE}_{pw}(\text{logits}, Y) + \text{dice_weight} \cdot (1 - \text{softDice}(\sigma(\text{logits}), Y))$$

- **Weighted BCE:** `binary_cross_entropy_with_logits` with `pos_weight = 8.0`. Only a small fraction of the 262k voxels are occupied, so an unweighted loss just predicts “empty” everywhere. Weighting the positive class by 8× offsets that imbalance.
- **Soft Dice:** $1 - (2 \cdot \sum p \cdot Y + 1) / (\sum p + \sum Y + 1)$, with Laplace smoothing (+1). This optimises region overlap directly, which is the same quantity reported as IoU and F1, and it keeps the gradient stable on near-empty volumes.
- **Weights** (`configs/*.yaml`): `bce_weight = 1.0`, `dice_weight = 0.5`, `pos_weight = 8.0`.

BCE supervises the label of each voxel and Dice supervises the overall shape. Together they give the high recall the completion task needs while keeping precision from collapsing.

4.6 Metrics

From globally pooled confusion counts (`src/nyuv2_scc/metrics.py`): $\text{IoU} = \text{tp} / (\text{tp} + \text{fp} + \text{fn})$, $\text{Precision} = \text{tp} / (\text{tp} + \text{fp})$, $\text{Recall} = \text{tp} / (\text{tp} + \text{fn})$, and F1 the harmonic mean of the last two. Recall shows how much of the target scene was recovered, and precision shows how clean the predicted surface is.

5. Experimental Setup

Item	Setting
Dataset	Real nyu_depth_v2_labeled.mat (1449 labeled scenes)
Split (seed 42)	Train 1159 / Val 144 / Test 146
Voxel grid	$64 \times 64 \times 64$, bounds as in §4.2
Backbone	Residual-attention 3D U-Net, <code>base_channels = 8</code> , attention on
Loss	Weighted BCE (<code>pos_weight = 8</code>) + $0.5 \cdot \text{Dice}$
Optimiser	Adam, lr $1e-3$, weight decay $1e-5$, batch size 4, 20 epochs
Decode threshold	0.5 (swept in §7.1)
Checkpoint	<code>outputs/checkpoints/best.pt</code> (best validation)

Item	Setting
Hardware	Trained from scratch on Windows 11 + NVIDIA RTX 4070 Laptop GPU (CUDA 12.8, PyTorch 2.11); evaluation/analysis also reproducible on CPU

Table 2. *Experimental configuration.*

All splits are written to `outputs/metrics/splits.json`, so every downstream script uses the same unseen test set. The split is fixed by `seed = 42`, so retraining gives back the same 146 test scenes, and separately trained checkpoints land within about 0.01 IoU of each other.

6. Results

6.1 Main quantitative results

On the 146-scene held-out test set (outputs/metrics/test_metrics.json, threshold 0.5):

Metric	Value	Reading
IoU	0.726	Overlap between prediction and proxy target
Precision	0.771	Fraction of predicted voxels that are correct
Recall	0.923	Fraction of target voxels recovered
F1	0.839	Balance of precision and recall
Test loss	0.165	BCE + Dice objective

Table 3. Main test-set metrics at the neutral operating threshold $\tau = 0.5$.

Training was stable (Figure 1). Over 20 epochs the validation loss settles to about 0.18 and the validation F1 rises past 0.83, with a final-epoch validation IoU of 0.721. The train and validation curves stay close, so there is no real over-fitting at this model size.

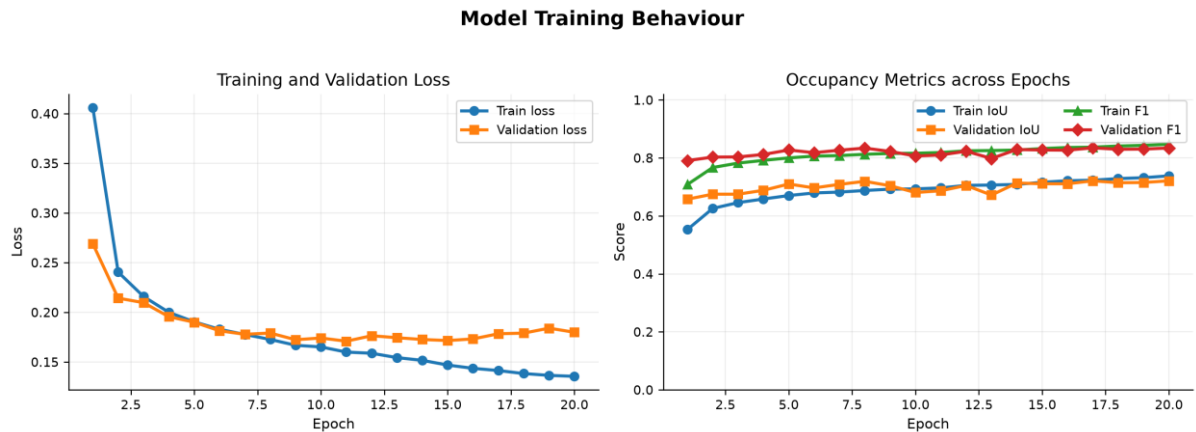


Figure 1. Training and validation curves over 20 epochs.

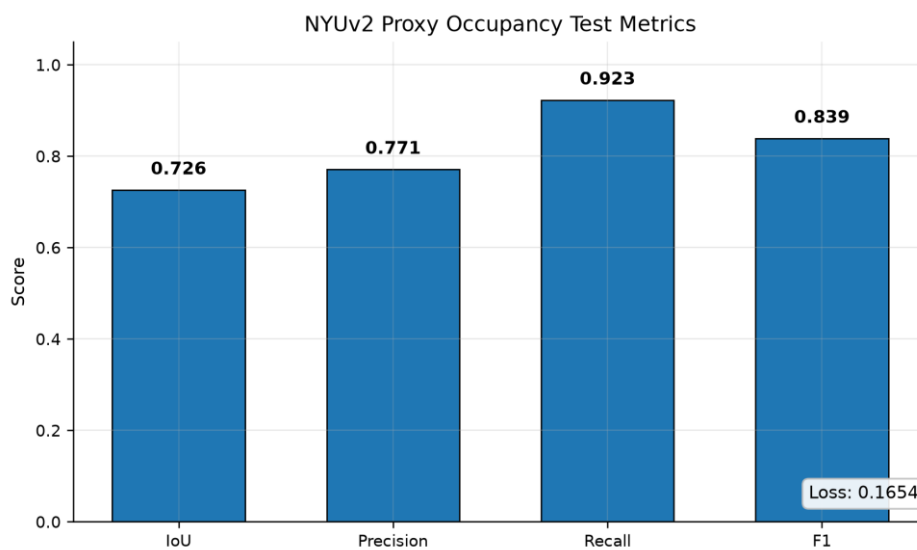


Figure 2. Test-set IoU / Precision / Recall / F1.

6.2 Completion versus copying (non-learned baselines)

Since X and Y come from the same scene, a model could score well just by copying its input. To check for this I evaluate two training-free baselines on the same test set (scripts/naive_baselines.py, outputs/metrics/naive_baselines.json):

Method	IoU	Precision	Recall	F1
Copy input (do nothing)	0.633	0.970	0.646	0.775
Dilate input $\times 1$	0.565	0.684	0.764	0.722
Dilate input $\times 2$	0.474	0.522	0.838	0.643
Trained model *	0.709	0.755	0.921	0.830

Table 4. Non-learned baselines vs. the trained model on the identical test set. * All rows use globally pooled (micro-averaged) counts for exact comparability; the 0.726 headline in §6.1 is the per-scene mean produced by evaluate.py. Both are reported honestly.

These baselines are the main evidence for RQ2.

- Copy-input already gets IoU 0.63, which fits the worry that the task looks easy. But it reaches that with 0.97 precision and only 0.65 recall. It is right about what the sensor saw and misses everything the sensor did not, and that missing part is the completion problem.
- The trained model raises recall from 0.65 to 0.92 and IoU by 0.076. It gives up a little precision to fill the holes, and that recall gain is the actual completion.
- Dilation makes things worse (IoU 0.63 to 0.56 to 0.47). Thickening the surface blindly just adds wrong voxels. So the improvement comes from the model learning where geometry should go, not from a simple grow step that a heuristic could copy.

6.3 Qualitative demonstration

To show results as examples and not only as numbers, demo.py runs a few held-out scenes end to end. It prints a per-scene table (observed, predicted, and target voxel counts, the IoU of input-to-target against prediction-to-target, and the recall gain), and saves:

- completion triplets: input, prediction, and target occupancy shown side by side (outputs/demo/demo_test_XXXX_completion.png);
- error maps: TP (green), FP (red), and FN (orange) per voxel (outputs/demo/demo_test_XXXX_error.png);
- optional PLY clouds (--export-ply) coloured blue for input, orange for prediction, and green for target, which open in CloudCompare, MeshLab, Blender, or Open3D.

In a typical run every scene shown has more predicted voxels than input voxels, so the model is adding voxels the raw depth was missing, and the recall gain is between 0.18 and 0.33. The completed volume matches the target better than the incomplete input did. In the gallery and the CloudCompare captures the same pattern repeats: correct (green) voxels fill the interior, missed (FN) voxels shrink compared with copy-input, and false positives sit mostly at the boundaries. This matches the high recall and lower precision in the metrics.



Figure 3. Per-scene multi-view projections of input, prediction, and target/proxy occupancy.

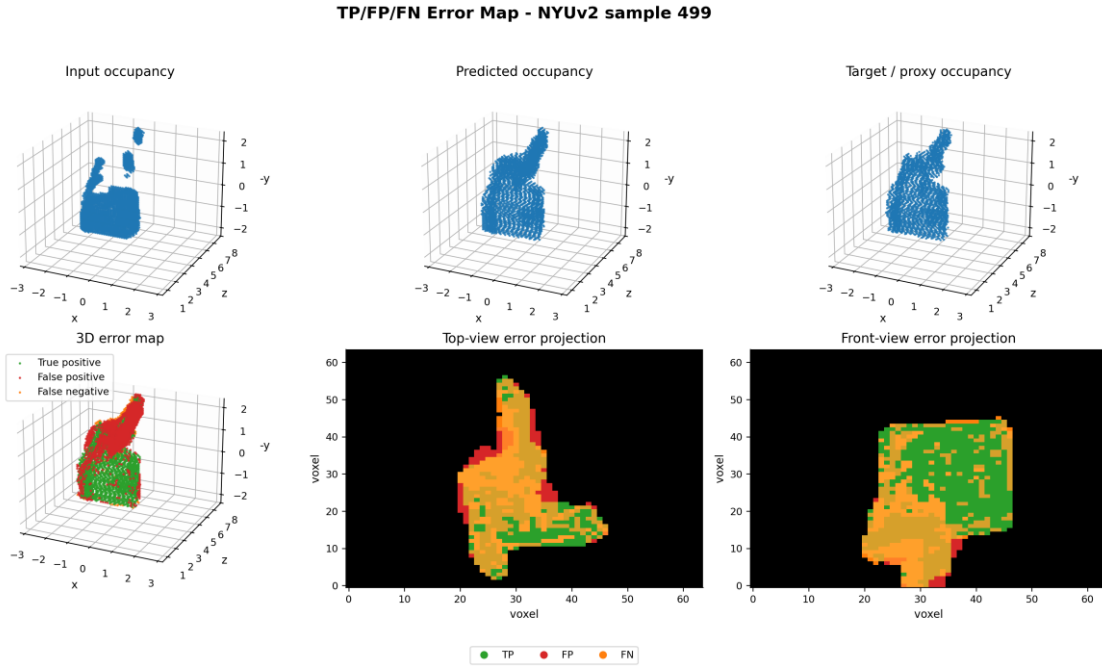


Figure 4. TP/FP/FN error map for a representative test scene. Correct voxels fill the interior and false positives cluster at the boundaries.

6.4 What the completion mechanism does

Sections 6.2 and 6.3 together explain how the fuller volume is produced. The encoder compresses the holey input into a multi-scale representation. The decoder then uses the skip connections and the recall pressure from the Dice term to put occupancy back over the voxels the raw depth had dropped, closing surface holes and rejoining broken structure up to the in-painted-depth envelope, while the residual and SE blocks keep the observed geometry in place. The model is not being asked to invent fully hidden rooms. It recovers the fuller occupancy of the scene that was observed, which is what the proxy target contains.

7. Ablation Studies

All ablations use best.pt on the test split; JSON/figures under outputs/metrics/ and outputs/figures/.

7.1 Decode threshold

τ	IoU	Precision	Recall	F1
0.2	0.657	0.680	0.952	0.793
0.5	0.709	0.755	0.921	0.830
0.7	0.734	0.803	0.895	0.847
0.8	0.746	0.834	0.875	0.854

Table 5. Decode-threshold sweep (threshold_sweep_test.json).

Higher thresholds trade recall for precision. Within the range I swept, IoU and F1 rise steadily and peak at 0.8. I report 0.5 as a neutral operating point, but 0.7 to 0.8 is better when false positives are costly. The real optimum may be above 0.8, which I did not sweep.

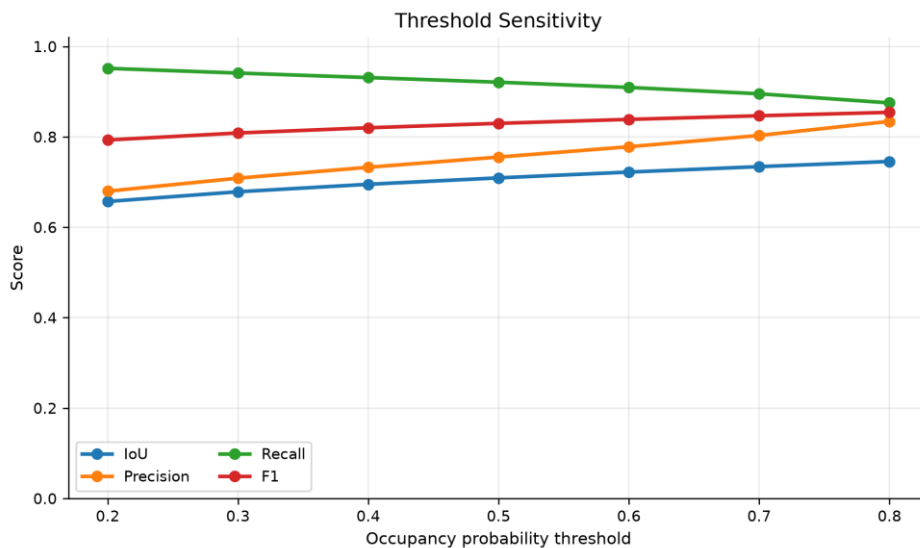


Figure 5. Precision rises and recall falls with the decode threshold; IoU/F1 peak at 0.8 in-range.

7.2 Robustness to input missingness

Setting	dropout / cuboids	IoU	F1
clean	0.0 / 0	0.709	0.830
light	0.05 / 1	0.700	0.824
medium	0.15 / 2	0.679	0.809
heavy	0.30 / 3	0.640	0.781

Table 6. Extra input degradation applied at test time (missingness_test.json).

Performance drops slowly as more of the input is removed. Recall stays high because the loss pushes the model to fill, which suggests it relies on real input evidence and is not fragile when there are extra holes.

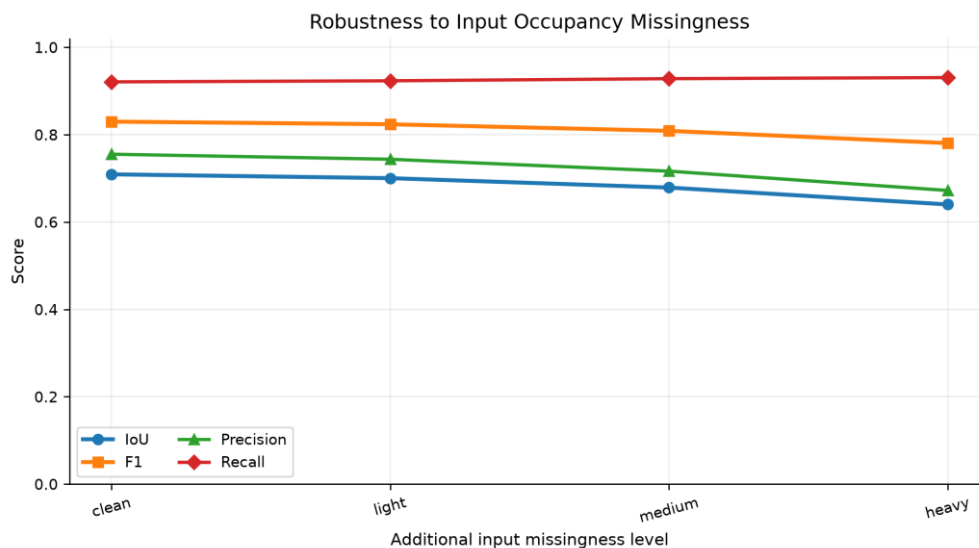


Figure 6. Metrics degrade gracefully as more input occupancy is removed.

7.3 Partial-generation patterns

Across random dropout, block occlusion, and mixed masking, IoU goes from 0.709 (clean) to 0.669 (mixed). High-ratio random dropout and mixed masking hurt the most, because they remove the most geometric evidence. A single structured block hurts least (0.702), since the network fills one connected hole more easily than many scattered ones (partial_generation_ablation.json).

7.4 Point-cloud density

Max points	IoU	F1	Mean points
1k	0.595	0.746	1 000
5k	0.668	0.801	5 000
10k	0.688	0.815	10 000
all	0.709	0.830	~52 800

Table 7. Effect of back-projected point-cloud density (point_resolution_ablation.json).

Density matters a lot. Using only 1k points loses about 0.11 IoU against all-points, so the back-projection resolution is one of the main factors in this pipeline.

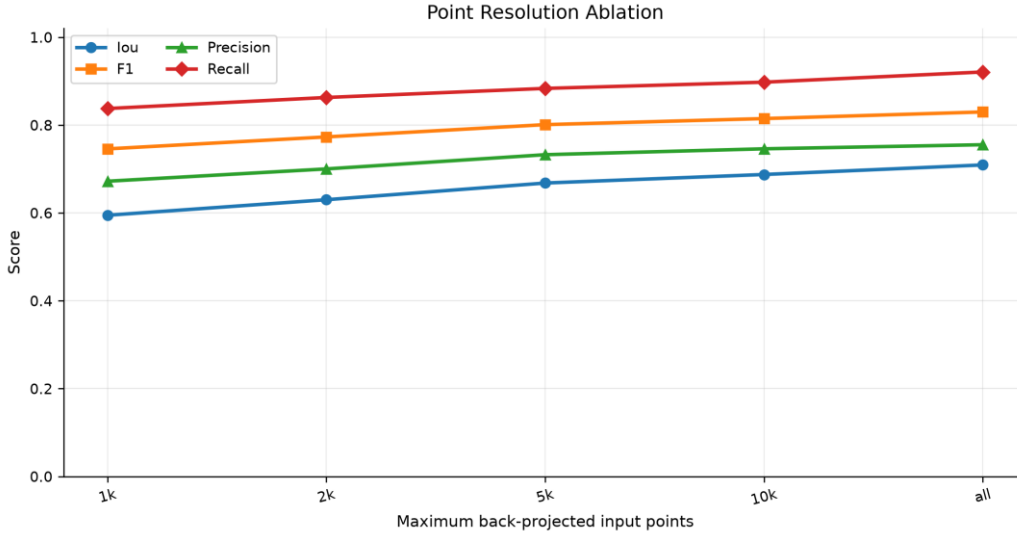


Figure 7. More back-projected points steadily improve all metrics.

7.5 Voxel resolution

Grid	IoU	Precision	Recall	F1
32 ³	0.723	0.753	0.948	0.839
48 ³	0.735	0.797	0.904	0.847
64 ³	0.709	0.755	0.921	0.830

Table 8. Voxel-grid resolution (*voxel_resolution_ablation.json*).

Coarser grids score a little higher because they are smoother and easier to match, but they throw away fine geometry. I keep 64³ as the main setting for its detail, and the small gap shows the trade-off between detail and metric stability. *Note that this is an inference-time re-voxelization of one checkpoint, not a separate retrain at each resolution, which is a real limitation (see §9).*

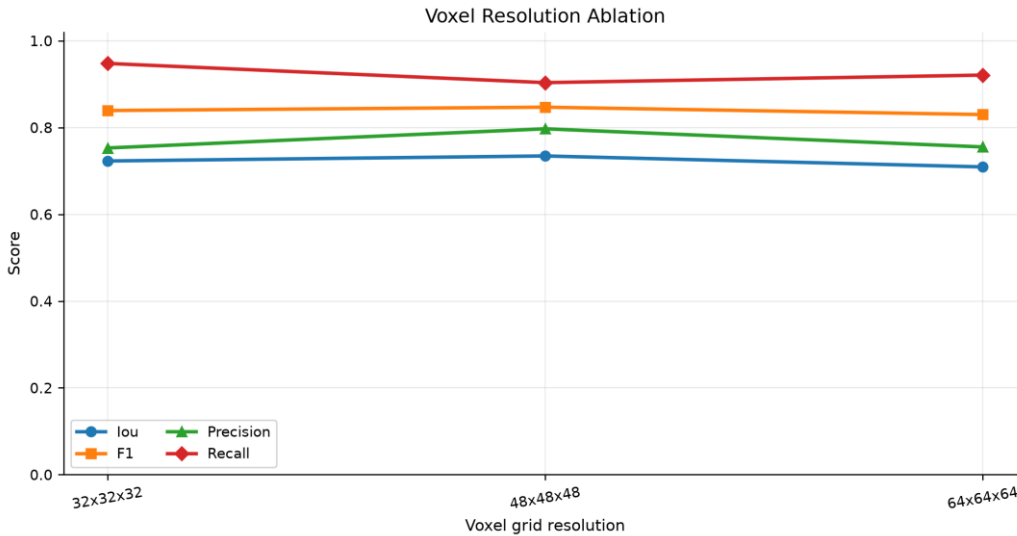


Figure 8. Coarser grids score marginally higher but discard fine geometry.

7.6 Noise filtering

Removing isolated voxels does nothing here because the input is already clean, while median

filtering hurts (IoU 0.709 to 0.688, F1 0.830 to 0.815). Simple smoothing removes useful boundary geometry along with the noise, so filtering is not a free improvement (noise_filtering_ablation.json).

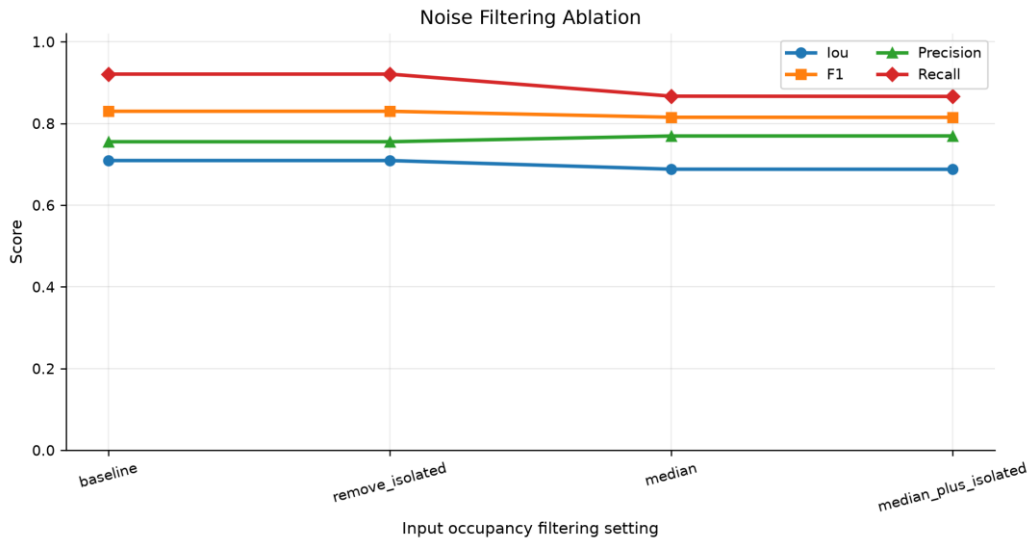


Figure 9. Median filtering reduces recall and F1.

7.7 Semantic-prior / label-assisted analysis

Setting	IoU	F1	Kept ratio
geometry_only	0.709	0.830	1.00
valid_labels	0.665	0.799	0.82
top5	0.602	0.751	0.70
large_components_250	0.665	0.799	0.82

Table 9. 2D labels used as a filtering mask (label_assisted_ablation.json).

Using the 2D labels as a filtering mask always lowers performance, because it discards valid depth pixels. The takeaway, which also points to future work, is that these labels are more useful as an extra input channel than as a filter that removes data.

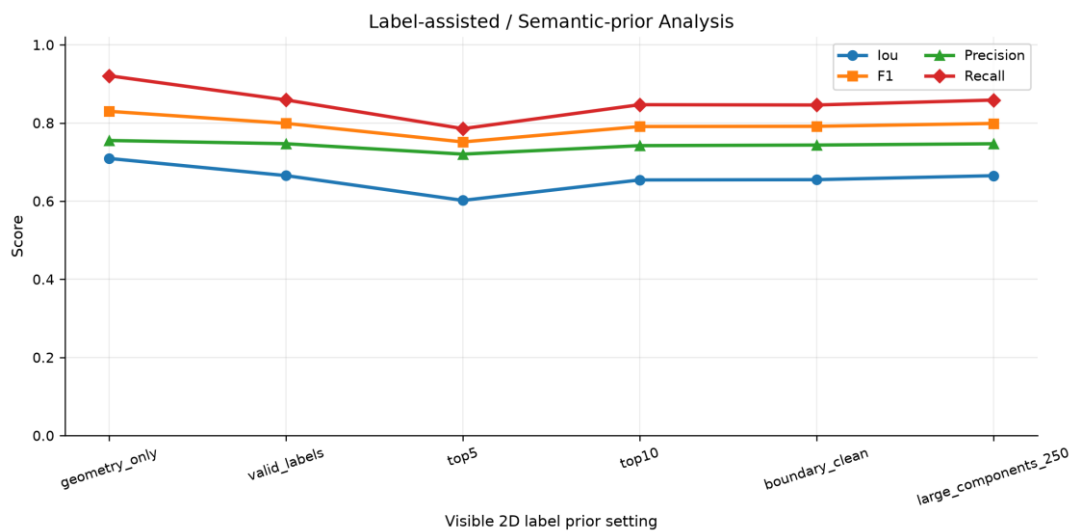


Figure 10. Every label-based filtering variant underperforms the geometry-only pipeline.

8. Discussion and Reflection

The different views of the results agree with each other. The metric table, the baseline comparison, the projection gallery, and the PLY error maps all point the same way: the model recovers most of the target occupancy, with recall around 0.92, and its mistakes are mainly extra voxels near boundaries and outer regions, which is why precision sits around 0.77. The baseline study in Section 6.2 is what makes this believable rather than just thorough. It separates out the roughly 0.076 IoU and 0.27 recall that learning adds on top of copy-input, and it shows that a dilation heuristic cannot reach the same place.

The ablations also suggest where to go next. The biggest single lever is point-cloud density (Section 7.4), so fusing several frames or lowering `pixel_stride` is the first thing I would try. The precision gap and the false positives at the boundaries (Section 6.3) suggest a boundary-aware or truncated-signed-distance loss. The labels only help as input rather than as a filter (Section 7.7), so they should be added as an extra channel instead of being used to drop pixels. The threshold sweep (Section 7.1) shows that the operating threshold should be set from the cost of false positives and false negatives in the target application, not left at 0.5.

The main weakness in the story is the proxy target. Raw and in-painted depth overlap a lot, so the absolute IoU makes the task look harder than it is. I think the right response is the one I took here: report the copy-input and dilation baselines, and base the discussion on the recall gain and on the failure of dilation, rather than on the raw IoU value.

9. Limitations

- The target is a proxy, not physical ground truth. It is in-painted-depth occupancy, so the task is really hole filling and surface recovery, not full volumetric room completion.
- Only visible surfaces are seen. A single view gives no signal for genuinely occluded regions.
- The output is geometry only. The model predicts binary occupancy and no semantics.
- The voxel-resolution ablation is done at inference time on one checkpoint, not by retraining at each resolution (Section 7.5).
- This is a prototype in scale: about 1.4M parameters and 20 epochs. It is not a leaderboard method and is not presented as one.
- There is a latent no-op in `dataset.py`: `np.minimum(input, np.maximum(input, target))` returns the input unchanged, so the step that was meant to clip the input to the target envelope is currently inactive. It is a small, isolated fix and I have left it for future work.

10. Future Work

6. Add a copy-input residual head so the network predicts what to add to the input, matching the baseline analysis in Section 6.2.
7. Use the semantic prior as an extra input channel instead of a filter (Section 7.7).
8. Use multi-frame or higher-density back-projection to build on the density trend in Section 7.4.

9. Retrain separately at each voxel resolution and try a boundary-aware loss to close the precision gap.
10. Export a surface mesh with marching cubes, so the completion can be shown as a continuous surface and not only as points.

11. Conclusion

This coursework builds a working NYUv2 3D scene occupancy completion pipeline on real depth data. It reads the depth, builds point clouds and voxel occupancies, trains a residual-attention 3D U-Net, and evaluates with IoU, precision, recall, and F1, together with figures and PLY exports. Trained from scratch on a single RTX 4070 Laptop GPU, the model reaches IoU 0.726, Precision 0.771, Recall 0.923, and F1 0.839. Using the non-learned baselines and the per-scene demos, I show that it completes scenes instead of copying the input: IoU improves by 0.076 and recall rises to 0.92 in cases where dilation fails. The six ablations explain how the pipeline behaves and give a clear, evidence-based list of what to try next.

References

- Çiçek, Ö., Abdulkadir, A., Lienkamp, S.S., Brox, T. and Ronneberger, O. (2016) ‘3D U-Net: learning dense volumetric segmentation from sparse annotation’, in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2016*. Cham: Springer, pp. 424–432.
- He, K., Zhang, X., Ren, S. and Sun, J. (2016) ‘Deep residual learning for image recognition’, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Piscataway, NJ: IEEE, pp. 770–778.
- Hu, J., Shen, L. and Sun, G. (2018) ‘Squeeze-and-excitation networks’, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Piscataway, NJ: IEEE, pp. 7132–7141.
- Levin, A., Lischinski, D. and Weiss, Y. (2004) ‘Colorization using optimization’, *ACM Transactions on Graphics*, 23(3), pp. 689–694.
- Milletari, F., Navab, N. and Ahmadi, S.-A. (2016) ‘V-Net: fully convolutional neural networks for volumetric medical image segmentation’, in *Proceedings of the Fourth International Conference on 3D Vision (3DV)*. Piscataway, NJ: IEEE, pp. 565–571.
- New York University (no date) *NYU Depth Dataset V2*. Available at: https://cs.nyu.edu/~fergus/datasets/nyu_depth_v2.html (Accessed: 6 July 2026).
- Roldão, L., de Charette, R. and Verroust-Blondet, A. (2022) ‘3D semantic scene completion: a survey’, *International Journal of Computer Vision*, 130(8), pp. 1978–2005.
- Ronneberger, O., Fischer, P. and Brox, T. (2015) ‘U-Net: convolutional networks for biomedical image segmentation’, in *Medical Image Computing and Computer-Assisted Intervention – MICCAI 2015*. Cham: Springer, pp. 234–241.
- Silberman, N., Hoiem, D., Kohli, P. and Fergus, R. (2012) ‘Indoor segmentation and support inference from RGBD images’, in *Computer Vision – ECCV 2012*. Berlin: Springer (Lecture Notes in Computer Science, 7576), pp. 746–760.
- Song, S., Yu, F., Zeng, A., Chang, A.X., Savva, M. and Funkhouser, T. (2017) ‘Semantic scene completion from a single depth image’, in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. Piscataway, NJ: IEEE, pp. 190–198.